
Preliminary Design Description - Elevator Project TTK4145

Marius August Devold: mariusdevo@gmail.com
Sondre Håmpland Myhre: sondrh@stud.ntnu.no
Eirik Heir: eiriheir@stud.ntnu.no

Group number: 12 Wednesdays 08:15 – 12:00, workstation 13

February, 2026

1 Strategy

The brain of the distributed system and main functionality will revolve around the OrderSync-module that can be seen in Figure 1. The operation of a single elevator is based on the handout-code "elevator". Each elevator will run as independent state machines, handling cab orders and door operations locally. Cab orders are written to file as a backup against power loss or crashes. To satisfy the button-light contract, each order is stored on disk before the corresponding cab light is activated. After a restart, orders are restored from file and the cab lights are set accordingly. If a node is experiencing network failure, it will operate as a single elevator, it will store the hall orders on disk as backup to fulfill the button light contract.

Hall lights mirror active orders and are updated by scanning the order matrix to enable the corresponding lamps. Hall lights are only activated after an order has been confirmed by other live peers via heartbeat exchange, ensuring the order survives a single crash and can be reassigned. OrderSync handles hall lights, while LocalSingleElevator controls cab lights.

The main problem to solve is then how to realize multiple elevators and coordinate them. The OrderSync module is responsible for distributing hall orders through a hallOrderMatrix (see Figure 2). Each matrix-element includes a version field for conflict resolution, where newer updates overwrite older ones. An incrementing counter per event ensures deterministic tie-breaking, making the system idempotent and commutative. This means packet loss can be mitigated by transmitting messages multiple times.

2 Network topology

We will use a peer-to-peer topology, as it fits our order-handling strategy. In this setup, the hallOrderMatrix acts as a distributed shared state, enabling the system to tolerate faults such as network loss, power loss, and software crashes in a single elevator. All network messages are broadcasted to all nodes, making UDP a sufficient protocol due to its broadcast support. We use the same port for sending and receiving, however when the simulation runs on a single computer, each elevator process has listen to a unique UDP port.

3 Programming language

Go is well suited for distributed systems, like a simultaneous elevator system, because the language has built-in support for concurrency through goroutines and channels. It also has built-in error handling

through the standard library, we will use the log package. On top of this, Go is fast and efficient, it compiles fast to machine code and has automatic memory management. As a result, Go handles asynchronous events, network communication, and state machines effectively. These properties make Go a natural choice for this project, allowing us to achieve high robustness while keeping the system complexity as low as possible.

4 System modules

The system is composed of multiple ElevatorNodes, where each node is controlling an elevator. Each ElevatorNode consists of the following modules: ElevIO, OrderSync, TransportUDP, PeerMonitor and LocalSingleElevator (see Figure 1). ElevIO provides a hardware abstraction layer towards the elevator-server or simulator (we will use the handed out code), OrderSync coordinates global order distribution and fault handling across the network, TransportUDP implements peer-to-peer communication over UDP, and PeerMonitor detects failed peers using heartbeat timeouts. PeerMonitor declares a node dead when several expected heartbeats is missed, and after a node is declared dead OrderSync releases its orders, and reassigns.

A Watchdog component is included for each ElevatorNode to monitor the process, and restart the process if it halts or crashes.

Appendiks A

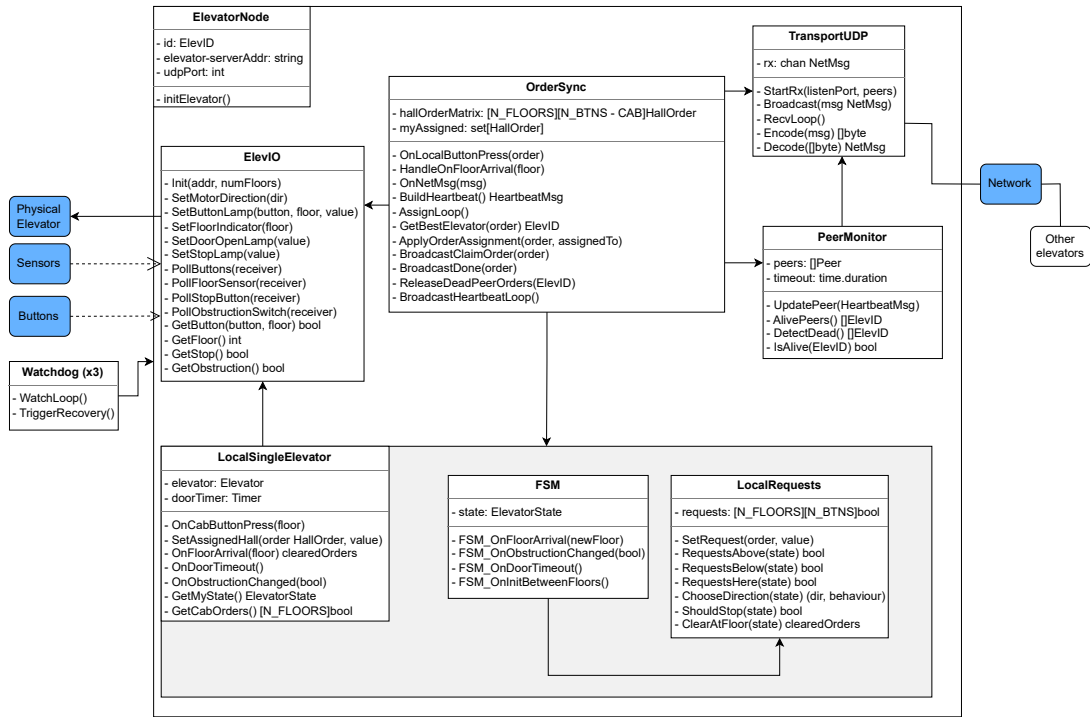


Figure 1: Component-/module diagram. Solid arrows represent API calls between components, dashed arrows represent signal/data flow from external entities, and solid lines represent communication links. The blue boxes represent external entities, while white boxes represent software components. ElevatorNode represents an elevator controller process, and the components inside ElevatorNode are the modules executed within that process.

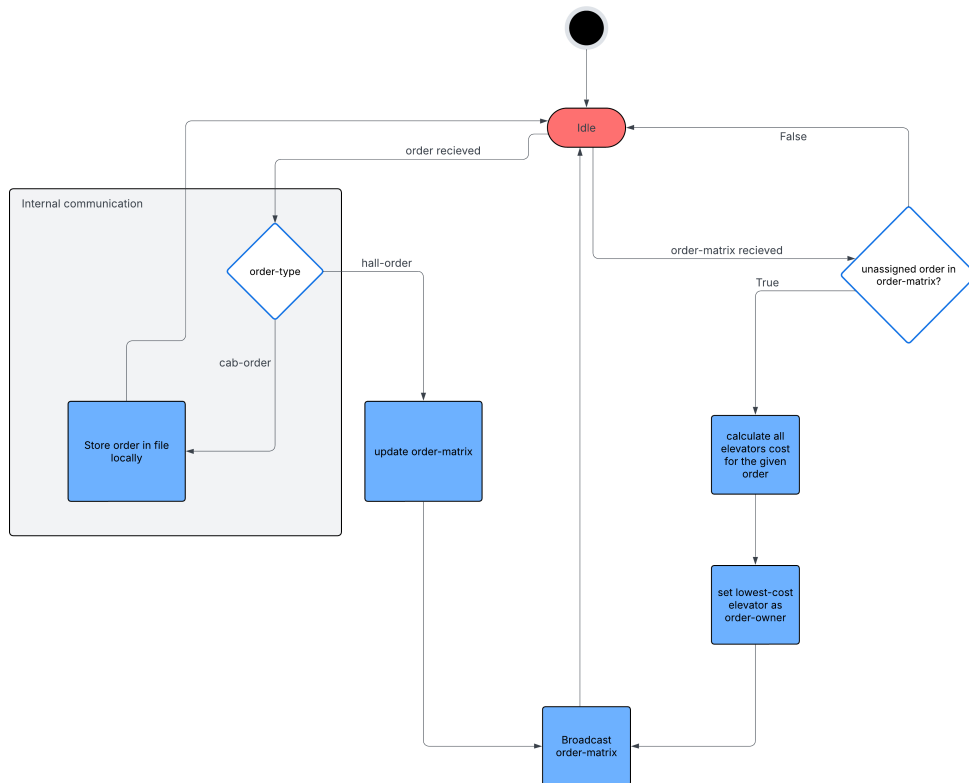


Figure 2: Diagram describing how orders are handled in a single elevator. The leftmost area describes how button pushes and recieved orders are handled, while the rightmost side represents recieved order-matrixes and how unassigned orders are given ownership.